

(A.1) Diferencias entre Apache Spark y Apache Hadoop

1. Modelo de Procesamiento: Spark procesa los datos en memoria (RAM), eliminando cuellos de botella de entrada/salida. Hadoop (MapReduce) procesa en disco, leyendo y escribiendo tras cada etapa.
2. Rendimiento y Velocidad: Gracias al procesamiento en memoria y la optimización de grafos de ejecución (DAG), Spark es hasta significativamente más rápido que Hadoop en cargas HPC y algoritmos iterativos.
3. Tipos de Carga de Trabajo: Spark es un motor unificado para procesamiento por lotes (*batch*), *streaming* en tiempo real, *Machine Learning* y análisis de grafos. Hadoop está diseñado casi exclusivamente para tareas *batch* pesadas y diferidas.
4. Abstracción de Datos: Spark gestiona los datos en memoria mediante estructuras tolerantes a fallos (RDDs, DataFrames, Datasets). Hadoop fragmenta y almacena datos físicamente en bloques a través del disco (HDFS).
5. Tolerancia a Fallos: Ante un fallo de nodo, Spark regenera los datos perdidos recalculando el historial de operaciones (linaje). Hadoop depende de la redundancia física, replicando el mismo bloque de datos en varios discos.
6. Spark ofrece mayor flexibilidad en gestores de clúster, mientras que Hadoop se basa principalmente en YARN.

(A.2) ¿Cómo se instala Apache Spark?

Tal y como aparece en el tutorial:

```
In [58]: #spark_version = "3.5.8"
#hadoop_version = "3"

#!apt-get update
#!apt-get install openjdk-8-jdk-headless -qq # Install JVM v8
#!wget -O spark-{spark_version}-bin-hadoop{hadoop_version}.tgz -q https://downlo
#!tar xf spark-{spark_version}-bin-hadoop{hadoop_version}.tgz # Unzip
#import os
#os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-8-openjdk-amd64"
#os.environ["SPARK_HOME"] = f"/content/spark-{spark_version}-bin-hadoop{hadoop_v
```

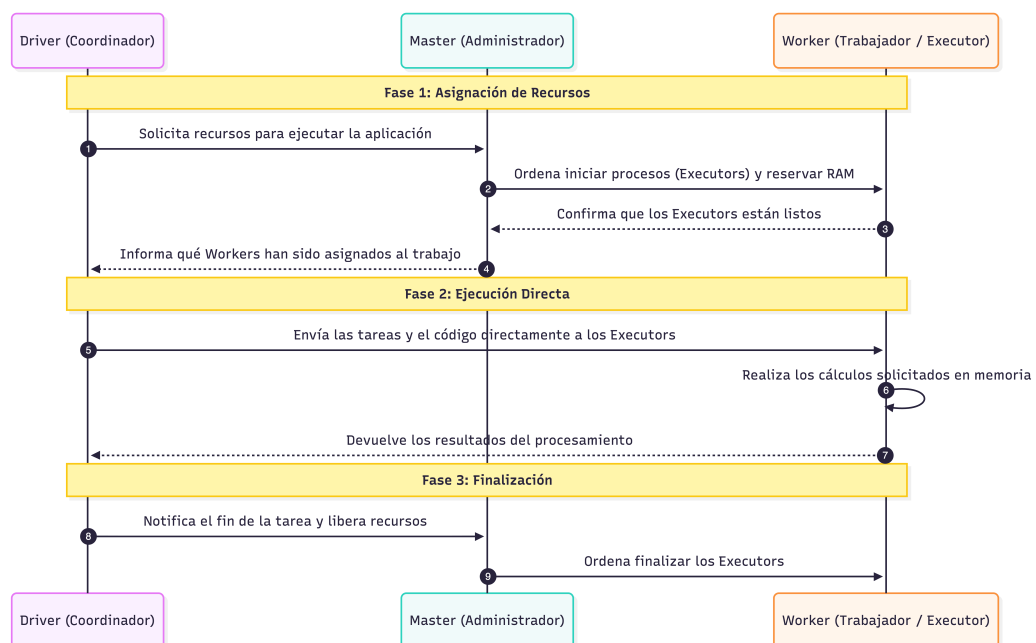
```
In [59]: #%pip install pyspark>=4.0.0
```

Para Mac es algo distinto. Hay que hacerlo desde un instalador como Homebrew. Además necesita algún motor jdk, pero esto ya estaba instalado en la máquina, por lo que no ha supuesto un problema.

(A.3) ¿Qué elementos componen un cluster de Apache Spark?

Los elementos que componen un clúster de Apache Spark son el driver, el nodo master y los nodos worker. El driver se encarga de entender el problema y solicitar los recursos necesarios para resolverlo al master. El master se encarga de reservar los recursos necesarios (en forma de trabajadores, nodos computacionales) para resolver el problema concreto. Es entonces cuando el driver da la orden de comenzar la ejecución, enviando tareas a los workers, quienes realizan los cálculos y devuelven los resultados. Cuando el proceso ha terminado, el driver notifica al master, y éste ordena la interrupción de la ejecución a los workers.

El siguiente diagrama detalla intuitivamente este proceso:



(A.4) ¿Qué es un RDD?

Un RDD es la estructura de datos más básica con la que trabaja Spark. En general, tiene 4 características innegociables:

- Es una colección de elementos particionada y distribuida entre los nodos del clúster. No impone ningún esquema ni tipo concreto: puede contener objetos Python, tuplas, diccionarios, etc. Es la abstracción de datos más primitiva de Spark, sobre la que se construyen APIs de más alto nivel como los DataFrames de Spark (propios de pyspark.sql), que son una evolución posterior y más optimizada, pero conceptualmente independiente de los DataFrames de pandas.
- Es distribuido. Este conjunto de datos se divide en fragmentos que se almacenan de forma distribuida entre los distintos nodos (workers) del sistema distribuido.

- Es inmutable. Si se quiere hacer un cambio, nunca se hace sobre el original, siempre sobre copias completamente nuevas.
- Es tolerante a fallos. De fallar algún nodo, Spark es capaz de deshacer el camino y entender como se llegó a los datos que se han perdido por estar en ese nodo. de esta forma los regenera en un nodo sano.

En función del lenguaje de programación, los RDD pueden ser o no heterogéneos. En realidad en Python, Apache corre sobre Java. Pero al ser Python dinámico en tipado hace que los RDD sean heterogéneos, mientras que en Java siempre son homogéneos.

```
In [60]: import os
from pyspark.sql import SparkSession

# definir explícitamente las rutas de entorno para macOS
os.environ['SPARK_HOME'] = '/opt/homebrew/Cellar/apache-spark/4.1.1'

spark = SparkSession.builder.appName("PruebaEntornoMac").getOrCreate()
sc = spark.sparkContext

datos = [x for x in range(5)]
datos.append('abcd')
rdd = sc.parallelize(datos)
print(f"Datos procesados correctamente: {rdd.collect()}")

spark.stop()
```

26/03/31 12:40:26 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.

Datos procesados correctamente: [0, 1, 2, 3, 4, 'abcd']

Efectivamente, los datos no tienen por qué ser homogéneos en *Python*.

(A.5) Particiones de un RDD

El número de particiones de un RDD es lo que intuitivamente podemos pensar que es: el número de fragmentos lógicos en los que Spark divide los datos para tenerlos listos para el procesamiento en paralelo. En concreto, para esta práctica, al no poder ejecutar el notebook desde un cluster de ordenadores, Apache encapsulará las particiones de acuerdo con el número de cores que tenga nuestra máquina (esto sucede en el caso ideal, pues podría hacerlo erróneamente). Esto supone un trade-off: un número muy bajo de particiones podría forzar al sistema operativo o a Apache a no utilizar todos los cores de la máquina. Sin embargo, de emplear un número alto, el sistema operativo podría sobrecargarse (pues Apache lanza las particiones como hilos que podrían comerse toda la memoria en un tal caso).

Para hacer un número fijo de particiones podríamos, o bien llamar al `parallelize` con el número de particiones deseadas como argumento:

```
In [61]: spark = SparkSession.builder.appName("PruebaEntornoMac").getOrCreate()
sc = spark.sparkContext
```

```
datos = [x for x in range(5)]
datos.append('abcd')
rdd = sc.parallelize(datos, 10) # para 10 cores
print(f"Número de particiones: {rdd.getNumPartitions()}")

spark.stop()
```

Número de particiones: 10

O bien modificar el objeto con su método *repartition*:

```
In [ ]: spark = SparkSession.builder.appName("PruebaEntorno").getOrCreate()
sc = spark.sparkContext

datos = [x for x in range(5)]
datos.append('abcd')
rdd = sc.parallelize(datos)
# Para 5 cores, recordemos que rdd es inmutable, hay que crear un nuevo objeto:
rdd_new = rdd.repartition(5)
print(f"Número de particiones: {rdd_new.getNumPartitions()}")

spark.stop()
```

Número de particiones: 5

O bien hacerlo de forma eficiente usando el método *coalesce*:

```
In [ ]: spark = SparkSession.builder.appName("PruebaEntorno").getOrCreate()
sc = spark.sparkContext

datos = [x for x in range(5)]
datos.append('abcd')
rdd = sc.parallelize(datos)
# Para 4 cores, recordemos que rdd es inmutable, hay que crear un nuevo objeto:
rdd_new = rdd.coalesce(4)
print(f"Número de particiones: {rdd_new.getNumPartitions()}")

spark.stop()
```

Número de particiones: 4

Este método reduce de la forma óptima (según Spark) el número de particiones. No sirve para aumentarlas, solo para disminuirlas.

Nótese que en todas ellas se llama al método *getNumPartitions* que permite conocer el número de particiones.

(A.6) SparkSQL vs RDD

Para comprobar si podemos obtener una columna en formato RDD, seleccionaremos una columna arbitraria de un DataFrame y la intentaremos pasar a RDD:

```
In [64]: spark = SparkSession.builder.appName("ComparativaRendimiento").getOrCreate()

datos = [(float(i),) for i in range(1000000)]
df = spark.createDataFrame(datos, ["valor"])
```

```

rdd_columna = df.select("valor") # seleccionamos la columna "valor"
rdd_columna = rdd_columna.rdd      # La pasamos a formato original, rdd
rdd_columna = rdd_columna.map(lambda fila: fila[0]) # rompemos las tuplas que ex

print(type(rdd_columna), type(df))

```

```

<class 'pyspark.core.rdd.PipelinedRDD'> <class 'pyspark.sql.classic.dataframe.DataFrame'>

```

Efectivamente podemos pasarla a RDD, el formato original de Spark.

```

In [65]: import time
import pyspark.sql.functions as F

# Media manual

inicio_manual = time.time()

suma_y_cuenta = rdd_columna.map(lambda x: (x, 1)).reduce(lambda a, b: (a[0] + b[0], a[1] + b[1]))
media_manual = suma_y_cuenta[0] / suma_y_cuenta[1]

tiempo_manual = time.time() - inicio_manual

# Media RDD

inicio_rdd = time.time()

media_rdd = rdd_columna.mean()

tiempo_rdd = time.time() - inicio_rdd

# Media API Spark

inicio_sql = time.time()

media_sql = df.select(F.mean("valor")).collect()[0][0]

tiempo_sql = time.time() - inicio_sql

print("-" * 50)
print(f"Media Manual (Map/Reduce): {media_manual} | Tiempo: {tiempo_manual:.4f} segundos")
print(f"Media RDD (función mean) : {media_rdd} | Tiempo: {tiempo_rdd:.4f} segundos")
print(f"Media API SparkSQL      : {media_sql} | Tiempo: {tiempo_sql:.4f} segundos")
print("-" * 50)

spark.stop()

```

```

26/03/31 12:40:31 WARN TaskSetManager: Stage 0 contains a task of very large size (1079 KiB). The maximum recommended task size is 1000 KiB.
26/03/31 12:40:32 WARN TaskSetManager: Stage 1 contains a task of very large size (1079 KiB). The maximum recommended task size is 1000 KiB.
26/03/31 12:40:32 WARN TaskSetManager: Stage 2 contains a task of very large size (1079 KiB). The maximum recommended task size is 1000 KiB.

```

```

-----
Media Manual (Map/Reduce): 499999.5 | Tiempo: 0.8065 segundos
Media RDD (función mean) : 499999.5 | Tiempo: 0.5350 segundos
Media API SparkSQL      : 499999.5 | Tiempo: 0.1690 segundos
-----

```

Como era de esperar, los calculos de Spark son notablemente mejores que los manuales. Pero la razón por la que la API de Spark es más rápida es, principalmente, que no necesita traducir de la máquina virtual de Java JVM, sino que realiza el cálculo desde su motor interno. En suma, el optimizador Catalyst es capaz de tomarse su tiempo, analizando la petición y optimizándola matemáticamente, mientras que el RDD ejecuta las instrucciones paso a paso.

(A.7) Carga y creación de datos con RDDs y con la API de Spark

Podemos gestionar ingesta de datos en RDDs en un contexto de Spark *sc* de distintas formas. En general lo alimentaremos con datos no estructurados o colecciones básicas:

- *sc.parallelize()*: como hemos observado antes, ingiere colecciones básicas y las distribuye de forma transparente. Podemos encargar el uso de distintos números de particiones, como vimos anteriormente.
- Archivos de texto plano usando *sc.textFile()* (tolera .csv o .txt).
- Archivos que ya han sido serializados por Spark (formato binario): *sc.objectFile()*.
- Archivos intrínsecos de Hadoop (normalmente organizados en k-v) *sc.sequenceFile()* ó *sc.hadoopFile()*.
- Para leer múltiples archivos: *sc.wholeTextFiles()*

Por su parte, la API de alto nivel (SparkSQL) permite lecturas más sofisticadas y óptimas de estructuras de datos concretas usando DataFrames:

- Los archivos *.parquet* son el sello de máxima eficiencia de compresión y velocidad de lectura en la industria. Por eso, la API de Spark tenía que ofrecer lectura de este tipo de archivos mandatoriamente. *spark.read.parquet()* ofrece esta lectura nativa de archivos de esta extensión.
- Puede leer archivos semi-estructurados como JSON o CSV deduciendo automáticamente el tipo de datos de cada columna. Ofrece *spark.read.json()* y *spark.read.csv()*.
- Puede solicitar conexiones estándar a BBDD MySQL, PostgreSQL u Oracle con *spark.read.jdbc()*.
- Puede ingerir incluso datos de algunas Data Warehouses como Apache Hive usando *spark.read.table()*.
- Para integrar múltiples fuentes de datos se usa *read.format*

(A.8) Guardado de datos con RDDs y con la API de Spark

La evolución tecnológica de las operaciones de guardado reflejan una muy parecida a las de carga y creación que vimos anteriormente. El de los RDDs es bastante rudimentario y delega responsabilidad del formato al programador, mientras que la API de alto nivel (SparkSQL) proporciona mayor control, si no absoluto, acerca del formato de guardado de los datos.

RDDs:

- Guardado en formato de texto plano: guarda cada elemento en una línea, *saveAsTextFile()*.
- Archivos serializados: guarda los datos en un formato binario específico de Java. Es más eficiente pero de nada sirve fuera de éste entorno, *saveAsObjectFile()*.
- Archivos de secuencia: es el formato original de Hadoop, el k-v que presentamos antes, pero binarizado. Le pasa un poco como al anterior formato, fuera de un entorno Hadoop, te sirve más bien de poco por eficiente que sea. *saveAsSequenceFile()*.
- Archivos nativos de Hadoop: *saveAsHadoopFile()*. Permite mayor poder de configuraciones y sistemas de almacenamiento de Hadoop sin perder demasiada eficiencia, pero, de nuevo, el mismo problema.

SparkSQL:

- Formatos orientados a columnas (el estado del arte de la computación de altas prestaciones): permiten guardar datos en *.parquet* *df.write.parquet()* o *.orc* *df.write.orc()*.
- Aunque penalicen el rendimiento, los datos en JSON o CSV son perfectos para ser consumidos por APIs (RESTful, entre otras) o para ser leídos por humanos. Se usa *df.write.json()*.
- BBDD externas, como vimos anteriormente, usando conexiones estándar: *df.write.jdbc()*.
- Data Warehouses: *df.write.saveAsTable()*.